Chapitre 5 — Fonctions, séparation du code et tests

Prérequis : chapitres 1 à 4 · **Exercices :** `E05_Fonctions_separation_tests`

Objectifs

- découper un programme en fonctions ;
- choisir entre passage par valeur, par référence et par référence constante ;
- regrouper des données liées dans une `struct` ;
- répartir le code en plusieurs fichiers (`.h` et `.cpp`) et le compiler ;
- tester une fonction : cas de test, valeur attendue, valeur obtenue, verdict.

Sommaire

- Chapitre 5 — Fonctions, séparation du code et tests
  - Objectifs
  - Sommaire
  - 1. Déclarer et définir une fonction
  - 2. Paramètres, arguments et valeur de retour
  - 3. Passage des paramètres par valeur
  - 4. Portée des variables
  - 5. Les références
  - 6. La référence constante
  - 7. Regrouper des données avec `struct`
  - 8. Séparer le code
    - 8.1 Une fonction, une responsabilité
    - 8.2 Répartir le code en plusieurs fichiers
    - 8.3 Compiler un programme en plusieurs fichiers
  - 9. Tester une fonction
  - Erreurs fréquentes
  - Synthèse

1. Déclarer et définir une fonction

Une **fonction** est un bloc d'instructions nommé qui réalise une tâche précise. Elle évite les répétitions, donne un nom explicite à un traitement et permet de le **tester** isolément.

```
int additionner(int a, int b) {    // type de retour, nom, parametres
    return a + b;                  // corps
}
```

Dans cet exemple :

- `int` est le type de retour ;
- `additionner` est le nom de la fonction ;
- `a` et `b` sont les paramètres ;
- `return` renvoie le résultat de l'addition.

Le compilateur doit connaître une fonction **avant** son appel. Soit on la définit au-dessus de `main()`, soit on écrit son **prototype** (sa déclaration) au-dessus et sa définition plus bas :

```cpp
#include <iostream>

int additionner(int a, int b);   // prototype (declaration)

int main() {
    std::cout << "Resultat : " << additionner(12, 8) << '\n';
    return 0;
}

int additionner(int a, int b) {  // definition
    return a + b;
}
```

Résultat :

```
Resultat : 20
```

2. Paramètres, arguments et valeur de retour

Les **paramètres** sont les variables déclarées dans la définition de la fonction. Les **arguments** sont les valeurs transmises lors de son appel, **dans le même ordre**.

```cpp
int multiplier(int a, int b) {
    return a * b;
}

int resultat{multiplier(4, 5)};   // a recoit 4, b recoit 5 : resultat vaut 20
```

- `a` et `b` sont les paramètres ;
- `4` et `5` sont les arguments ;
- `resultat` reçoit la valeur retournée, ici `20`.

> **Remarque :** une fonction `void` ne renvoie rien ; elle peut utiliser `return;` pour s'arrêter plus tôt.

3. Passage des paramètres par valeur

Par défaut, la fonction reçoit une **copie** de l'argument.

```cpp
#include <iostream>

void modifier(int nombre) {
    nombre = 100;   // modifie la copie seulement
    std::cout << "Dans la fonction : " << nombre << '\n';
}

int main() {
    int valeur{10};

    modifier(valeur);
    std::cout << "Apres l'appel : " << valeur << '\n';

    return 0;
}
```

Résultat :

```
Dans la fonction : 100
Apres l'appel : 10
```

4. Portée des variables

Une variable déclarée dans une fonction ou dans un bloc `{ }` est **locale** : elle n'existe que jusqu'à la fin de ce bloc. Une variable **globale**, déclarée hors de toute fonction, est accessible partout.

> **Conseil :** on **évite les variables globales modifiables** : n'importe quelle fonction peut les changer, ce qui rend le programme difficile à comprendre et à tester.

5. Les références

Une **référence** est un autre nom pour une variable existante. Elle est initialisée à sa déclaration et désigne ensuite toujours la même variable.

```cpp
#include <iostream>

int main() {
    int valeur{10};
    int& reference{valeur};

    reference = 25;
    std::cout << valeur << '\n';

    return 0;
}
```

Résultat :

```
25
```

Un paramètre de type référence permet à la fonction d'agir **sur l'original** :

```cpp
#include <iostream>

void incrementer(int& nombre) {   // reference : agit sur l'original
    nombre++;
}

int main() {
    int compteur{10};

    incrementer(compteur);
    std::cout << compteur << '\n';

    return 0;
}
```

Résultat :

```
11
```

6. La référence constante

Passer un gros objet (`std::string`, `std::vector`, `struct`) **par valeur** le copie entièrement à chaque appel. Quand la fonction doit seulement le **lire**, on le passe par **référence constante** `const T&` : aucune copie, et le compilateur interdit toute modification.

```cpp
#include <iostream>
#include <string>
#include <vector>

// Lecture seule : reference constante, aucune copie du vecteur
double moyenne(const std::vector<double>& valeurs) {
    if (valeurs.empty()) {
        return 0.0;
    }
    double somme{0.0};
    for (double v : valeurs) {
        somme += v;
    }
    return somme / static_cast<double>(valeurs.size());
}

void afficherTitre(const std::string& titre) {
    std::cout << "== " << titre << " ==\n";
}

int main() {
    std::vector<double> mesures{21.5, 22.0, 23.5};

    afficherTitre("Releve du capteur T1");
    std::cout << "Moyenne : " << moyenne(mesures) << '\n';

    return 0;
}
```

Résultat :

```
== Releve du capteur T1 ==
Moyenne : 22.3333
```

- Dans `moyenne`, une instruction comme `valeurs.push_back(0.0);` provoquerait une **erreur de compilation** : la référence est constante.
- Une référence constante accepte aussi une valeur directe, comme le texte `"Releve du capteur T1"` ; une référence simple `std::string&` la refuserait.

Besoin de la fonction	Passage	Exemple
Lire une petite valeur (`int`, `double`, `char`, `bool`)	Par valeur	`int carre(int n)`
Modifier l'argument	Référence `T&`	`void incrementer(int& n)`
Lire un objet volumineux (`std::string`, `std::vector`, `struct`)	Référence constante `const T&`	`double moyenne(const std::vector<double>& v)`

7. Regrouper des données avec `struct`

Une `struct` regroupe plusieurs données liées sous un même nom. On accède aux champs avec le point.

```cpp
#include <iostream>
#include <string>

struct Mesure {
    std::string capteur{};
    double valeur{};
};

int main() {
    Mesure m{"T1", 23.5};

    std::cout << m.capteur << " : " << m.valeur << '\n';

    return 0;
}
```

Résultat :

```
T1 : 23.5
```

Une `struct` se transmet à une fonction comme tout objet volumineux, par référence constante pour la lire :

```cpp
void afficherMesure(const Mesure& m) {
    std::cout << m.capteur << " : " << m.valeur << '\n';
}
```

En C++, `struct` et `class` sont presque identiques. La `class`, qui ajoute le contrôle d'accès aux données, est le point de départ du cours de POO.

8. Séparer le code

8.1 Une fonction, une responsabilité

Un programme lisible sépare les **entrées-sorties** (saisie, affichage) des **traitements** (calculs, décisions). Une fonction de calcul qui n'affiche rien et ne lit rien au clavier peut être réutilisée ailleurs (autre programme, carte Arduino) et **testée** automatiquement (§9).

```cpp
// Traitement : aucun affichage, aucune saisie
double celsiusVersFahrenheit(double celsius) {
    return celsius * 9.0 / 5.0 + 32.0;
}

// Entree-sortie : uniquement l'affichage
void afficherConversion(double celsius) {
    std::cout << celsius << " C = " << celsiusVersFahrenheit(celsius) << " F\n";
}
```

8.2 Répartir le code en plusieurs fichiers

Dès qu'un programme grandit, on regroupe les fonctions par thème dans des **modules**. Un module comprend deux fichiers :

Fichier	Contenu	Rôle
`conversion.h` (en-tête)	**Déclarations** : prototypes, constantes `constexpr`, définitions de `struct` et de `class`	Ce que les autres fichiers doivent connaître pour utiliser le module
`conversion.cpp` (source)	**Définitions** des fonctions	Le code qui réalise les traitements

Les fichiers qui utilisent le module incluent seulement son **en-tête**.

```cpp
// Fichier : conversion.h
#ifndef CONVERSION_H
#define CONVERSION_H

constexpr double ZERO_ABSOLU_CELSIUS{-273.15};

double celsiusVersFahrenheit(double celsius);
double celsiusVersKelvin(double celsius);
bool temperaturePhysique(double celsius);

#endif
```

```cpp
// Fichier : conversion.cpp
#include "conversion.h"

double celsiusVersFahrenheit(double celsius) {
    return celsius * 9.0 / 5.0 + 32.0;
}

double celsiusVersKelvin(double celsius) {
    return celsius - ZERO_ABSOLU_CELSIUS;
}

bool temperaturePhysique(double celsius) {
    return celsius >= ZERO_ABSOLU_CELSIUS;
}
```

```cpp
// Fichier : main.cpp
#include <iostream>
#include "conversion.h"

int main() {
    double t{25.0};

    if (temperaturePhysique(t)) {
        std::cout << celsiusVersFahrenheit(t) << " F, "
                  << celsiusVersKelvin(t) << " K\n";
    }

    return 0;
}
```

Résultat :

```
77 F, 298.15 K
```

- `#include "conversion.h"`, avec des **guillemets**, cherche le fichier dans le dossier du projet. Les chevrons `< >` sont réservés aux bibliothèques installées (bibliothèque standard).
- `conversion.cpp` inclut aussi son propre en-tête : le compilateur vérifie ainsi que les définitions correspondent aux prototypes.
- Les lignes `#ifndef` / `#define` / `#endif` forment une **garde d'inclusion** : si l'en-tête est inclus deux fois dans un même fichier (directement ou via un autre en-tête), son contenu n'est pris en compte qu'une fois. Beaucoup de compilateurs acceptent aussi `#pragma once`.
- On n'inclut **jamais** un fichier `.cpp`.

8.3 Compiler un programme en plusieurs fichiers

On donne à g++ **tous les fichiers** `.cpp` (jamais les `.h`) :

```
g++ -std=c++17 -Wall -Wextra main.cpp conversion.cpp -o conversion
```

g++ enchaîne trois étapes :

1. le **préprocesseur** remplace chaque `#include` par le contenu du fichier inclus ;
2. le **compilateur** traduit **chaque** `.cpp` séparément en fichier objet (`.o`) ;
3. l'**éditeur de liens** assemble les fichiers objets et vérifie que chaque fonction appelée est définie **une seule fois**.

On peut aussi compiler séparément, ce qui évite de tout recompiler quand un seul fichier change :

```
g++ -std=c++17 -Wall -Wextra -c conversion.cpp
g++ -std=c++17 -Wall -Wextra -c main.cpp
g++ main.o conversion.o -o conversion
```

Message	Étape	Cause probable
`'celsiusVersKelvin' was not declared in this scope`	Compilation	En-tête non inclus, ou prototype absent
`undefined reference to 'celsiusVersKelvin(double)'`	Édition de liens	Fichier `.cpp` oublié dans la commande, ou fonction déclarée mais jamais définie
`multiple definition of 'celsiusVersKelvin(double)'`	Édition de liens	Fonction **définie** dans un `.h` inclus par plusieurs `.cpp`

> **Remarque :** la même organisation s'applique aux classes (cours de POO, chapitre 2) et aux bibliothèques Arduino : voir le TP guidé « Créer une bibliothèque Arduino » (document séparé `TP_Bibliotheque_Arduino.md`).

9. Tester une fonction

Une fonction qui compile n'est pas forcément correcte. **Tester**, c'est comparer ce que la fonction **produit** à ce qu'elle **devrait** produire, pour des entrées choisies.

Terme	Signification
Cas de test	Une entrée choisie et le résultat attendu pour cette entrée
Valeur attendue (*expected*)	Résultat correct, déduit du cahier des charges
Valeur obtenue (*actual*)	Résultat réellement renvoyé par la fonction
Verdict	**Réussi** (*OK, passed*) si obtenue = attendue, **échoué** (*failed*) sinon
Test unitaire	Test d'une seule fonction (ou classe), isolée du reste

On choisit des valeurs **ordinaires**, les **valeurs limites** (bornes incluses et premières valeurs hors bornes) et des valeurs **invalides**. Pour une plage de 0 à 100 incluses, les valeurs limites sont -1, 0, 100 et 101.

```cpp
#include <iostream>

// Fonction testee : vrai si la valeur est comprise entre 0 et 100 inclus
bool valeurValide(int valeur) {
    return valeur >= 0 && valeur <= 100;
}

// Fonction de test : compare l'obtenu a l'attendu et affiche le verdict
bool testerValeurValide(int entree, bool attendu) {
    bool obtenu{valeurValide(entree)};

    std::cout << std::boolalpha;
    if (obtenu == attendu) {
        std::cout << "OK     valeurValide(" << entree << ")\n";
        return true;
    }
    std::cout << "ECHEC  valeurValide(" << entree << ") : attendu "
              << attendu << ", obtenu " << obtenu << '\n';
    return false;
}

int main() {
    int nbEchecs{0};

    if (!testerValeurValide(50, true))   { nbEchecs++; }   // valeur ordinaire
    if (!testerValeurValide(0, true))    { nbEchecs++; }   // borne basse
    if (!testerValeurValide(100, true))  { nbEchecs++; }   // borne haute
    if (!testerValeurValide(-1, false))  { nbEchecs++; }   // juste sous la borne
    if (!testerValeurValide(101, false)) { nbEchecs++; }   // juste au-dessus

    std::cout << nbEchecs << " test(s) en echec\n";

    return 0;
}
```

Résultat :

```
OK     valeurValide(50)
OK     valeurValide(0)
OK     valeurValide(100)
OK     valeurValide(-1)
OK     valeurValide(101)
0 test(s) en echec
```

Si la fonction avait été écrite avec `valeur < 100`, seul le test de la borne 100 aurait échoué : d'où l'intérêt des valeurs limites.

> **Méthode — analyser un échec :** un test échoué signale un **écart** ; il reste à savoir si l'erreur est dans la **fonction** ou dans le **test** (attendu mal calculé, mauvaise unité, donnée mal saisie). Exemple : un test de `convertirEnVolts(3300.0)` indique « attendu 33, obtenu 3.3 ». Or 3 300 mV valent bien 3,3 V : c'est l'**attendu** qui est faux. Avant d'accuser le code, on recalcule l'attendu à la main, dans la **même unité**.

Deux outils complémentaires :

- les calculs en `double` sont approchés (`0.1 + 0.2 == 0.3` est faux) : on compare avec une tolérance, par exemple `std::fabs(obtenu - attendu) < 1e-9` (en-tête `<cmath>`) ;

- `assert(condition)` (en-tête `<cassert>`) arrête le programme si la condition est fausse. C'est pratique pour une vérification rapide, mais il est désactivé par l'option `-DNDEBUG` : il ne remplace pas les contrôles d'un programme.

```
#include <cassert>

bool valeurValide(int valeur) {
    return valeur >= 0 && valeur <= 100;
}

int main() {
    assert(valeurValide(0));
    assert(!valeurValide(101));
    return 0;
}
```

Les tests de classes sont approfondis dans le cours de POO (chapitre 7).

Erreurs fréquentes

Erreur	Correction
Appel avant la déclaration	Prototype au-dessus de `main()` ou dans un `.h` inclus
Arguments dans le mauvais ordre	Relire la signature
Passage par valeur alors qu'on veut modifier	Paramètre `T&`
Grosse chaîne ou gros vecteur passé par valeur	`const T&`
`std::string&` pour un paramètre seulement lu	`const std::string&` (accepte aussi `"texte"`)
`return` oublié dans une fonction non `void`	Un `return` sur chaque chemin (g++ avertit)
Fonction **définie** dans un `.h`	Prototype dans le `.h`, définition dans le `.cpp`
`.cpp` oublié dans la commande de compilation	`undefined reference` : ajouter le fichier
Seules des valeurs « faciles » testées	Tester les valeurs limites
Comparaison de deux `double` avec `==`	Comparer avec une tolérance

Synthèse

- Une fonction a un type de retour, un nom et des paramètres ; elle doit être déclarée (prototype) avant son appel.
- Passage par valeur : copie. Par référence `T&` : agit sur l'original. Par référence constante `const T&` : lecture sans copie d'un gros objet.
- Une `struct` regroupe des données liées ; on accède aux champs avec le point.
- Un module = un `.h` (déclarations, garde d'inclusion) + un `.cpp` (définitions). On compile tous les `.cpp`, jamais les `.h`.
- `undefined reference` = erreur d'édition de liens (fichier oublié ou fonction non définie).
- Tester = comparer obtenu et attendu, valeurs limites comprises ; en cas d'échec, vérifier aussi l'attendu.